

Fair & Unbiased Assessment System

Database Design Document

V 2.0

By

1	Abdul Rehman	NUM-BSCS-2024-01
2	Junaid Gondal	NUM-BSCS-2024-28
3	Ahmad Ali	NUM-BSCS-2024-02



Department of Computer Sciences
Namal University
Mianwali, Pakistan
Submission Date : 14th June, 2026

REVISION HISTORY

Date	Version	Description	Approved by
21/06/26	V 2.0	Updated the document according to Milestone 3 by adding ERD-to-relational schema conversion, functional dependencies, normalization up to 3NF, final relations, and implementation planning sections.	Group Members
15/05/26	V 1.3	Finalized the ERD report by updating relationship names, cardinalities, primary key identification, entity attributes, business rules, references, and AI appendix section.	Group Members
12/05/26	V 1.2	Introduced Program, Batch, Session, and Section entities and improved normalization decisions according to university workflow.	Ma'am Asiya

TABLE OF CONTENTS

CHAPTER 1: PROJECT OVERVIEW

1.1. INTRODUCTION

1.2. PROBLEM STATEMENT

1.3. PROJECT OBJECTIVES

1.4. GITHUB REPOSITORY LINK

CHAPTER 2: CONCEPTUAL DATABASE DESIGN

2.1. ENTITY

2.2. DATA DICTIONARY

2.3. BUSINESS RULES

2.4. ENTITY RELATIONSHIP DIAGRAM

CHAPTER 3: LOGICAL DATABASE DESIGN

3.1. RELATIONAL SCHEMA

3.1.1. ENTITY-TO-TABLE MAPPING AND KEY IDENTIFICATION

3.1.2. RELATIONSHIP REPRESENTATION

3.1.3. FINAL RELATIONAL SCHEMA

3.2. FUNCTIONAL DEPENDENCIES

3.3. NORMALIZATION

3.3.1. FIRST NORMAL FORM (1NF)

3.3.2. SECOND NORMAL FORM (2NF)

3.3.3. THIRD NORMAL FORM (3NF)

3.3.4. FINAL 3NF RELATIONS

CHAPTER 4: IMPLEMENTATION

4.1. LANGUAGE/Framework

4.2. DATABASE CONNECTIVITY

4.3. STORED PROCEDURES AND FUNCTIONS

4.4. INTERFACES

APPENDIX: AI USAGE AND PROMPTS

CHAPTER 1: PROJECT OVERVIEW

1.1. INTRODUCTION:

The purpose of this project is to design and implement a database-driven AI-Based Academic Assessment and Report Generation System for a university environment. The system is intended to manage academic activities such as user management, student enrollment, faculty teaching assignments, course registration, session-based assessment creation, student submissions, evaluation, and academic report generation.

The database design focuses on representing real university workflows in a structured manner. It supports program and batch management, academic sessions, semester levels, section handling, repeat-course enrollment, faculty assignments, registered courses, portal-controlled assessments, and preservation of historical academic records. The Entity Relationship Diagram (ERD/EERD) provides the conceptual foundation, while the relational schema converts that design into normalized relations suitable for database implementation.

This version of the document extends the previous ERD report by adding logical database design, relational schema conversion, functional dependencies, normalization decisions, and final 3NF relations required for implementation.

1.2. PROBLEM STATEMENT:

Traditional academic assessment systems often store student records, course details, assessments, and evaluation data in fragmented or manually maintained formats. Such systems may not properly manage relationships among students, programs, batches, sessions, sections, courses, teachers, and assessments. As a result, data redundancy, inconsistency, poor scalability, and difficulty in tracking historical academic records can occur.

Assessment evaluation and report generation are also time-consuming when handled manually. Faculty members must manage submissions, marks, feedback, and reports repeatedly for large classes. Without a normalized database structure, it becomes difficult to retrieve complete student histories, identify course registrations, connect students with teachers, manage repeat courses, and generate accurate academic reports.

Therefore, the proposed system requires a well-structured and normalized database design that can accurately represent university academic workflows, reduce redundancy, preserve historical information, and support future implementation through SQL scripts and a graphical user interface.

1.3. PROJECT OBJECTIVES:

The main objective of this project is to design and implement a normalized relational database for an AI-Based Academic Assessment and Report Generation System that supports academic management and report generation in a university environment.

- To identify all major entities required for the academic assessment system.
- To define suitable attributes, primary keys, and relationships for each entity.
- To model user specialization using a User supertype with Student and Faculty subtypes.
- To convert the conceptual ERD/EERD into a normalized relational schema.
- To identify functional dependencies and remove partial and transitive dependencies.

- To represent departments, programs, batches, batch-program combinations, sessions, semesters, sections, and courses accurately.
- To support student enrollment history across sessions, semesters, and sections.
- To support faculty teaching assignments and student registered courses.
- To support session-based assessment creation, portal status control, student submissions, evaluations, and report metadata.
- To prepare the database for implementation using DDL and DML SQL scripts.

1.4. GITHUB REPOSITORY LINK:

Please click the link for the GitHub repository. [[Link](#)]

CHAPTER 2: CONCEPTUAL DATABASE DESIGN

2.1. ENTITY:

The following entities represent the main concepts of the proposed system. Associative relations such as Enrollment, Teacher Assignment, Registered Course, and Batch Program are explained in the logical design section because they are mainly used to represent relationships and preserve normalized database structure.

Sr. No	Entity Name	Description
01	User	Stores common authentication and profile information for all system users including students, faculty members, and administrators.
02	Student	Stores student-specific academic information such as registration number, academic status, and batch-program membership.
03	Faculty	Stores faculty-specific information including designation and department association.
04	Department	Stores university department information and manages related programs, faculty members, and courses.
05	Program	Stores academic program details such as BS Computer Science, BS Electrical Engineering, BBA, and BS Mathematics.
06	Batch	Stores admission or graduation cohort information such as Batch 2028.
07	Semester	Stores academic semester level such as 1st semester, 2nd semester, up to 8th semester.
08	Session	Stores academic sessions or terms such as Spring 2026 and Fall 2026 with date ranges.
09	Section	Stores section information used to divide students into smaller academic groups.
10	Course	Stores course information including course code, course title, and credit hours.
11	Assessment	Stores session-specific assessment details such as quizzes, assignments, total marks, deadlines, portal status, and class report information.
12	Submission	Stores student-uploaded submissions for assessments through the registered course record, preventing duplicate submissions for the same assessment and course registration.
13	Evaluation	Stores evaluation results, marks, calculated percentage, status, feedback, and student report information for each submission.
14	User Phone	Stores multiple contact numbers associated with system users.
15	Activity Log	Stores system activity records for auditing and tracking operations.

2.2. DATA DICTIONARY:

The following data dictionary describes the important attributes of the conceptual entities. Foreign keys and bridge relations are documented in Chapter 3 as part of the relational schema conversion.

2.2.1. User

Sr. No	Name	Data Type	Constraint	Description
01	User_ID (Primary Key)	Integer	NOT NULL, PK	Unique identifier for the user.
02	First_Name	String	NOT NULL	First name of the user.

03	Last_Name	String	NOT NULL	Last name of the user.
04	Email	String	NOT NULL, UNIQUE	Email address of the user.
05	Password_Hash	String	NOT NULL	Stored password value used for authentication in the database.
06	Role	String	NOT NULL	Role of the user such as Admin, Faculty, or Student.
07	Gender	String	NULLABLE	Gender of the user.
08	Account_Status	String	NOT NULL	Status of the account such as Active or Inactive.
09	Created_At	DateTime	NOT NULL	Timestamp when the user account was created.

2.2.2. User Phone

Sr. No	Name	Data Type	Constraint	Description
01	Phone_ID (Primary Key)	Integer	NOT NULL, PK	Unique identifier for the phone record.
02	Phone_No	String	NOT NULL	Phone number of the user.

2.2.3. Student

Sr. No	Name	Data Type	Constraint	Description
01	Student_ID (Primary Key)	Integer	NOT NULL, PK	Unique identifier for the student.
02	Registration_No	String	NOT NULL, UNIQUE	Official registration number of the student.
03	Student_Status	String	NOT NULL	Current academic status such as Active, Course Repeat, Degree Pending, Graduated, or Dropped.

2.2.4. Faculty

Sr. No	Name	Data Type	Constraint	Description
01	Teacher_ID (Primary Key)	Integer	NOT NULL, PK	Unique identifier for the faculty member.
02	Designation	String	NOT NULL	Academic designation of the faculty member.

2.2.5. Department

Sr. No	Name	Data Type	Constraint	Description
01	Department_ID (Primary Key)	Integer	NOT NULL, PK	Unique identifier for the department.
02	Department_Name	String	NOT NULL	Full name of the department.
03	Department_Code	String	NOT NULL, UNIQUE	Short code representing the department.

2.2.6. Program

Sr. No	Name	Data Type	Constraint	Description
01	Program_ID (Primary Key)	Integer	NOT NULL, PK	Unique identifier for the academic program.
02	Program_Name	String	NOT NULL	Name of the academic program.
03	Program_Code	String	NOT NULL, UNIQUE	Short code for the program.
04	Degree_Level	String	NOT NULL	Level of degree such as Undergraduate or Graduate.

05	Total_Semester	Integer	NOT NULL	Total number of semesters in the program.
----	----------------	---------	----------	---

2.2.7. Batch

Sr. No	Name	Data Type	Constraint	Description
01	Batch_ID (Primary Key)	Integer	NOT NULL, PK	Unique identifier for the batch.
02	Batch_Name	String	NOT NULL	Name or label of the batch such as Batch 2028.
03	Batch_Status	String	NOT NULL	Current status of the batch.
04	Start_Year	Integer	NOT NULL	Year in which the batch started.

2.2.8. Semester

Sr. No	Name	Data Type	Constraint	Description
01	Semester_ID (Primary Key)	Integer	NOT NULL, PK	Unique identifier for the semester level.
02	Semester_No	Integer	NOT NULL	Semester number in the academic sequence.

2.2.9. Session

Sr. No	Name	Data Type	Constraint	Description
01	Session_ID (Primary Key)	Integer	NOT NULL, PK	Unique identifier for the academic session.
02	Session_Name	String	NOT NULL	Name of the session such as Spring 2026.
03	Academic_Year	String	NOT NULL	Academic year of the session.
04	Start_Date	Date	NOT NULL	Start date of the session.
05	End_Date	Date	NOT NULL	End date of the session.

2.2.10. Section

Sr. No	Name	Data Type	Constraint	Description
01	Section_ID (Primary Key)	Integer	NOT NULL, PK	Unique identifier for the section.
02	Section_Name	String	NOT NULL	Name or label of the section such as A or B.
03	Capacity	Integer	NOT NULL	Maximum number of students allowed in the section.

2.2.11. Course

Sr. No	Name	Data Type	Constraint	Description
01	Course_ID (Primary Key)	Integer	NOT NULL, PK	Unique identifier for the course.
02	Course_Code	String	NOT NULL, UNIQUE	Official code for the course.
03	Course_Title	String	NOT NULL	Full title of the course.
04	Credit_Hours	Integer	NOT NULL	Number of credit hours for the course.

2.2.12. Assessment

Sr. No	Name	Data Type	Constraint	Description
--------	------	-----------	------------	-------------

01	Assessment_ID (Primary Key)	Integer	NOT NULL, PK	Unique identifier for the assessment.
02	Teacher_Assignment_ID	Integer	NOT NULL, FK	References the teacher assignment for which the assessment is created.
03	Session_ID	Integer	NOT NULL, FK	References the academic session to keep the assessment session-specific.
04	Title	String	NOT NULL	Title of the assessment.
05	Description	Text	NULLABLE	Detailed description of the assessment.
06	Assessment_Type	String	NOT NULL	Type of assessment such as Quiz, Assignment, Lab Task, Project, or Exam.
07	Total_Marks	Decimal	NOT NULL	Maximum marks possible for the assessment.
08	Created_At	DateTime	NOT NULL	Timestamp when the assessment was created.
09	Deadline	Date	NOT NULL	Submission deadline for the assessment.
10	Portal_Status	String	NOT NULL	Controls whether students can upload submissions; values such as Open or Closed.
11	Class_Report_Path	String	NULLABLE	File path to the generated class report.
12	Class_Report_Generated_At	DateTime	NULLABLE	Timestamp when the class report was generated.
13	Class_Report_Status	String	NOT NULL	Current status of class report generation such as Not Generated or Generated.

2.2.13. Submission

Sr. No	Name	Data Type	Constraint	Description
01	Submission_ID (Primary Key)	Integer	NOT NULL, PK	Unique identifier for the submission.
02	Assessment_ID	Integer	NOT NULL, FK	References the assessment for which the file is submitted.
03	Registered_Course_ID	Integer	NOT NULL, FK	References the registered course so the submission is linked to the exact enrollment and teacher assignment.
04	File_Name	String	NOT NULL	Name of the submitted file.
05	File_Path	String	NOT NULL	Storage path of the submitted file.
06	Upload_Time	DateTime	NOT NULL	Timestamp when the file was uploaded.
07	Submission_Status	String	NOT NULL	Status of the submission such as Submitted.

2.2.14. Evaluation

Sr. No	Name	Data Type	Constraint	Description
01	Evaluation_ID (Primary Key)	Integer	NOT NULL, PK	Unique identifier for the evaluation.
02	Obtained_Marks	Decimal	NOT NULL	Marks obtained by the student.

03	Percentage	Decimal	NOT NULL	Percentage score calculated from obtained marks and assessment total marks.
04	Evaluated_At	DateTime	NOT NULL	Timestamp when evaluation was performed.
05	Evaluation_Status	String	NOT NULL	Status of evaluation such as Evaluated or Pending.
06	Feedback	Text	NULLABLE	Evaluation feedback generated by the system or teacher.
07	Student_Report_Path	String	NULLABLE	File path to the student report.
08	Student_Report_Generated_At	DateTime	NULLABLE	Timestamp when the student report was generated.
09	Student_Report_Status	String	NULLABLE	Status of student report generation.

2.2.15. Activity Log

Sr. No	Name	Data Type	Constraint	Description
01	Log_ID (Primary Key)	Integer	NOT NULL, PK	Unique identifier for the activity log entry.
02	Activity_Description	Text	NOT NULL	Detailed description of the activity.
03	Time	DateTime	NOT NULL	Timestamp when the activity occurred.

2.3. BUSINESS RULES:

- Each user in the system must have a unique account for authentication and access.
- A user can exist as a student, faculty member, or administrator according to the assigned role.
- A user can have multiple contact numbers stored in the system.
- Each student belongs to one batch-program combination throughout the degree duration.
- A batch represents a cohort label, such as Batch 2028, and may be connected to multiple programs through Batch_Program.
- A department can offer multiple academic programs, but each program belongs to one department.
- A department can offer multiple courses, but each course belongs to one department.
- A program may have multiple batch-program combinations across different batches.
- A batch-program combination may contain multiple students and multiple sections.
- Each section belongs to one batch-program combination and can contain multiple student enrollments.
- Each enrollment record belongs to one student and stores the student's section, session, semester, and enrollment status for historical tracking.
- A student can have multiple enrollment records across different academic sessions and semesters.
- A single enrollment can contain multiple registered courses.
- Each registered course links one enrollment with one teacher assignment.
- Each teacher assignment is associated with one faculty member, one course, one section, and one semester.

- #### 2.4. ENTITY RELATIONSHIP DIAGRAM:

[illegible]

Figure 1: Enhanced Entity Relationship Diagram

CHAPTER 3: LOGICAL DATABASE DESIGN

3.1. RELATIONAL SCHEMA

The transformation of the Entity-Relationship Diagram into a logical relational schema is the step where conceptual entities, attributes, and relationships are converted into implementable relational tables. In this design, strong entities are mapped directly to relations, subtype entities are mapped using primary-key/foreign-key relationships with the User relation, and many-to-many or historical academic relationships are represented using associative relations.

The final design is normalized up to Third Normal Form. Important corrections include introducing Batch_Program between Batch and Program, removing Current_Semester from Student because it can be derived from enrollment history, keeping section information inside Enrollment rather than Student, storing course selection through Registered_Course using Teacher_Assignment, and linking Submission through Registered_Course instead of Enrollment.

3.1.1. Entity-to-Table Mapping and Key Identification

User Relation

Attribute	Description
Relational Table	User
Primary Key	user_id
Foreign Keys	None
Columns	user_id, first_name, last_name, email, password_hash, role, gender, account_status, created_at
Explanation	The User entity is mapped to a strong relation that stores common profile and authentication information for all system users.

Student Relation

Attribute	Description
Relational Table	Student
Primary Key	student_id
Foreign Keys	user_id references User, batch_program_id references Batch_Program
Columns	student_id, user_id, batch_program_id, registration_no, student_status
Explanation	Student is a subtype/specialized user. Section is not stored in Student because section may change across sessions and is therefore stored in Enrollment.

Faculty Relation

Attribute	Description
Relational Table	Faculty
Primary Key	teacher_id
Foreign Keys	user_id references User, department_id references Department

Columns	teacher_id, user_id, department_id, designation
Explanation	Faculty is a subtype/specialized user and stores faculty-specific information.

Department Relation

Attribute	Description
Relational Table	Department
Primary Key	department_id
Foreign Keys	None
Columns	department_id, department_name, department_code
Explanation	Department directly maps to a relation and identifies academic divisions.

Program Relation

Attribute	Description
Relational Table	Program
Primary Key	program_id
Foreign Keys	department_id references Department
Columns	program_id, department_id, program_name, program_code, degree_level, total_semester
Explanation	Program belongs to one department and represents a degree program.

Batch Relation

Attribute	Description
Relational Table	Batch
Primary Key	batch_id
Foreign Keys	None
Columns	batch_id, batch_name, start_year, batch_status
Explanation	Batch stores cohort identity only. Program is not stored here because one batch year can include multiple programs.

Batch_Program Relation

Attribute	Description
Relational Table	Batch_Program
Primary Key	batch_program_id
Foreign Keys	batch_id references Batch, program_id references Program
Columns	batch_program_id, batch_id, program_id, expected_graduation
Explanation	This associative relation resolves the many-to-many relationship between Batch and Program.

Academic_Session Relation

Attribute	Description
-----------	-------------

Relational Table	Academic_Session
Primary Key	session_id
Foreign Keys	None
Columns	session_id, session_name, academic_year, start_date, end_date
Explanation	Academic_Session stores actual academic terms such as Spring 2026 and their date ranges.

Semester Relation

Attribute	Description
Relational Table	Semester
Primary Key	semester_id
Foreign Keys	None
Columns	semester_id, semester_no
Explanation	Semester stores only academic level such as 1st semester through 8th semester.

Section Relation

Attribute	Description
Relational Table	Section
Primary Key	section_id
Foreign Keys	batch_program_id references Batch_Program
Columns	section_id, batch_program_id, section_name, capacity
Explanation	Section belongs to a batch-program combination, allowing Section A of BSCS 2028 to be distinguished from Section A of another program or batch.

Course Relation

Attribute	Description
Relational Table	Course
Primary Key	course_id
Foreign Keys	department_id references Department
Columns	course_id, department_id, course_code, course_title, credit_hours
Explanation	Course belongs to one department and stores course code, title, and credit hours.

Teacher_Assignment Relation

Attribute	Description
Relational Table	Teacher_Assignment
Primary Key	teacher_assignment_id
Foreign Keys	teacher_id references Faculty, course_id references Course, semester_id references Semester, section_id references Section
Columns	teacher_assignment_id, teacher_id, course_id, semester_id, section_id, assigned_date
Explanation	This relation represents which teacher teaches which course in which semester and section.

Enrollment Relation

Attribute	Description
Relational Table	Enrollment
Primary Key	enrollment_id
Foreign Keys	student_id references Student, section_id references Section, session_id references Academic_Session, semester_id references Semester
Columns	enrollment_id, student_id, section_id, session_id, semester_id, is_repeat, enrollment_status, enrollment_start_date, enrollment_end_date
Explanation	Enrollment preserves student academic history for a particular session, semester, and section.

Registered_Course Relation

Attribute	Description
Relational Table	Registered_Course
Primary Key	registered_course_id
Foreign Keys	enrollment_id references Enrollment, teacher_assignment_id references Teacher_Assignment
Columns	registered_course_id, enrollment_id, teacher_assignment_id, registration_status
Explanation	Registered_Course records the courses included in a student enrollment through teacher assignments.

Assessment Relation

Attribute	Description
Relational Table	Assessment
Primary Key	assessment_id
Foreign Keys	teacher_assignment_id references Teacher_Assignment, session_id references Academic_Session
Columns	assessment_id, teacher_assignment_id, session_id, title, description, assessment_type, total_marks, created_at, deadline, portal_status, class_report_path, class_report_generated_at, class_report_status
Explanation	Assessment is created for a teacher assignment and academic session, and stores portal status, assessment details, and class report metadata.

Submission Relation

Attribute	Description
Relational Table	Submission
Primary Key	submission_id
Foreign Keys	assessment_id references Assessment, registered_course_id references Registered_Course

Columns	submission_id, assessment_id, registered_course_id, file_name, file_path, upload_time, submission_status
Explanation	Submission stores uploaded work for a specific assessment and registered course. The combination of assessment_id and registered_course_id is unique to avoid duplicate submissions.

Evaluation Relation

Attribute	Description
Relational Table	Evaluation
Primary Key	evaluation_id
Foreign Keys	submission_id references Submission
Columns	evaluation_id, submission_id, obtained_marks, percentage, evaluated_at, evaluation_status, feedback, student_report_path, student_report_generated_at, student_report_status
Explanation	Evaluation stores marks, percentage, feedback, and student report metadata for a submission.

User_Phone Relation

Attribute	Description
Relational Table	User_Phone
Primary Key	phone_id
Foreign Keys	user_id references User
Columns	phone_id, user_id, phone_no
Explanation	User_Phone stores multiple phone numbers for a user.

Activity_Log Relation

Attribute	Description
Relational Table	Activity_Log
Primary Key	log_id
Foreign Keys	user_id references User
Columns	log_id, user_id, activity_description, time
Explanation	Activity_Log records system activity associated with users.

3.1.2. Relationship Representation

Relationship	Type	Relational Representation
User to Student	One-to-one subtype	Student.user_id references User.user_id.
User to Faculty	One-to-one subtype	Faculty.user_id references User.user_id.
User to User_Phone	One-to-many	User_Phone.user_id references User.user_id.
User to Activity_Log	One-to-many	Activity_Log.user_id references User.user_id.
Department to Program	One-to-many	Program.department_id references Department.department_id.

Department to Course	One-to-many	Course.department_id references Department.department_id.
Department to Faculty	One-to-many	Faculty.department_id references Department.department_id.
Batch to Program	Many-to-many	Resolved using Batch_Program with batch_id and program_id.
Batch_Program to Student	One-to-many	Student.batch_program_id references Batch_Program.batch_program_id.
Batch_Program to Section	One-to-many	Section.batch_program_id references Batch_Program.batch_program_id.
Student to Enrollment	One-to-many	Enrollment.student_id references Student.student_id.
Enrollment to Registered_Course	One-to-many	Registered_Course.enrollment_id references Enrollment.enrollment_id.
Faculty/Course/Semester/Section to Teacher_Assignment	Many-to-one from each parent	Teacher_Assignment stores teacher_id, course_id, semester_id, and section_id.
Teacher_Assignment to Assessment	One-to-many	Assessment.teacher_assignment_id references Teacher_Assignment.teacher_assignment_id and Assessment.session_id references Academic_Session.session_id.
Assessment Registered_Course Submission and to	Many-to-one from each parent	Submission.assessment_id references Assessment.assessment_id and Submission.registered_course_id references Registered_Course.registered_course_id.
Submission to Evaluation	One-to-one	Evaluation.submission_id references Submission.submission_id and is unique.

3.1.3. Final Relational Schema

USER(User_ID PK, First_Name, Last_Name, Email UNIQUE, Password_Hash, Role, Gender, Account_Status, Created_At)

DEPARTMENT(

Department_ID PK, Department_Name, Department_Code UNIQUE)

PROGRAM(

Program_ID PK, Department_ID FK, Program_Name, Program_Code UNIQUE, Degree_Level, Total_Semester)

BATCH(

Batch_ID PK, Batch_Name, Start_Year, Batch_Status)

BATCH_PROGRAM(

Batch_Program_ID PK, Batch_ID FK, Program_ID FK, Expected_Graduation)

ACADEMIC_SESSION(

Session_ID PK, Session_Name, Academic_Year, Start_Date, End_Date)

SEMESTER(

Semester_ID PK, Semester_No)

FACULTY(

Teacher_ID PK, User_ID FK UNIQUE, Department_ID FK, Designation)

STUDENT(

Student_ID PK, User_ID FK UNIQUE, Batch_Program_ID FK, Registration_No UNIQUE, Student_Status)

USER_PHONE(

Phone_ID PK, User_ID FK, Phone_No)

ACTIVITY_LOG(

Log_ID PK, User_ID FK, Activity_Description, Time)

COURSE(

Course_ID PK, Department_ID FK, Course_Code UNIQUE, Course_Title, Credit_Hours)

SECTION(

Section_ID PK, Batch_Program_ID FK, Section_Name, Capacity)

TEACHER_ASSIGNMENT(

Teacher_Assignment_ID PK, Teacher_ID FK, Course_ID FK, Semester_ID FK, Section_ID FK, Assigned_Date)

ENROLLMENT(

Enrollment_ID PK, Student_ID FK, Section_ID FK, Session_ID FK, Semester_ID FK, Is_Repeat, Enrollment_Status, Enrollment_Start_Date, Enrollment_End_Date)

REGISTERED_COURSE(

Registered_Course_ID PK, Enrollment_ID FK, Teacher_Assignment_ID FK, Registration_Status)

ASSESSMENT(

Assessment_ID PK, Teacher_Assignment_ID FK, Session_ID FK, Title, Description, Assessment_Type, Total_Marks, Created_At, Deadline, Portal_Status, Class_Report_Path, Class_Report_Generated_At, Class_Report_Status)

SUBMISSION(

Submission_ID PK, Assessment_ID FK, Registered_Course_ID FK, File_Name, File_Path, Upload_Time, Submission_Status, UNIQUE(Assessment_ID, Registered_Course_ID))

EVALUATION(

Evaluation_ID PK, Submission_ID FK UNIQUE, Obtained_Marks, Percentage, Evaluated_At, Evaluation_Status, Feedback, Student_Report_Path, Student_Report_Generated_At, Student_Report_Status)

3.2. FUNCTIONAL DEPENDENCIES

Functional dependencies describe how attributes in each relation depend on the primary key or other candidate keys. These dependencies were used to verify that the final relations satisfy Third Normal Form.

Relation	Functional Dependencies
User	User_ID -> First_Name, Last_Name, Email, Password_Hash, Role, Gender, Account_Status, Created_At; Email -> User_ID
Department	Department_ID → Department_Name, Department_Code; Department_Code → Department_ID
Program	Program_ID → Department_ID, Program_Name, Program_Code, Degree_Level, Total_Semester; Program_Code → Program_ID
Batch	Batch_ID → Batch_Name, Start_Year, Batch_Status
Batch_Program	Batch_Program_ID → Batch_ID, Program_ID, Expected_Graduation; (Batch_ID, Program_ID) → Batch_Program_ID, Expected_Graduation
Academic_Session	Session_ID → Session_Name, Academic_Year, Start_Date, End_Date
Semester	Semester_ID → Semester_No; Semester_No → Semester_ID
Faculty	Teacher_ID → User_ID, Department_ID, Designation; User_ID → Teacher_ID
Student	Student_ID → User_ID, Batch_Program_ID, Registration_No, Student_Status; User_ID → Student_ID; Registration_No → Student_ID
User_Phone	Phone_ID → User_ID, Phone_No
Activity_Log	Log_ID → User_ID, Activity_Description, Time
Course	Course_ID → Department_ID, Course_Code, Course_Title, Credit_Hours; Course_Code → Course_ID
Section	Section_ID → Batch_Program_ID, Section_Name, Capacity
Teacher_Assignment	Teacher_Assignment_ID → Teacher_ID, Course_ID, Semester_ID, Section_ID, Assigned_Date
Enrollment	Enrollment_ID → Student_ID, Section_ID, Session_ID, Semester_ID, Is_Repeat, Enrollment_Status, Enrollment_Start_Date, Enrollment_End_Date
Registered_Course	Registered_Course_ID → Enrollment_ID, Teacher_Assignment_ID, Registration_Status
Assessment	Assessment_ID -> Teacher_Assignment_ID, Session_ID, Title, Description, Assessment_Type, Total_Marks, Created_At, Deadline, Portal_Status, Class_Report_Path, Class_Report_Generated_At, Class_Report_Status
Submission	Submission_ID -> Assessment_ID, Registered_Course_ID, File_Name, File_Path, Upload_Time, Submission_Status; (Assessment_ID, Registered_Course_ID) -> Submission_ID
Evaluation	Evaluation_ID -> Submission_ID, Obtained_Marks, Percentage, Evaluated_At, Evaluation_Status, Feedback, Student_Report_Path, Student_Report_Generated_At, Student_Report_Status; Submission_ID -> Evaluation_ID

3.3. NORMALIZATION

Normalization was applied to reduce redundancy and avoid insertion, update, and deletion anomalies. The final relational schema was checked step by step against First Normal Form, Second Normal Form, and Third Normal Form. This section first explains each normal form, then shows which relations already satisfied it, and finally explains the changes made to convert the remaining parts into proper normalized relations.

3.3.1. First Normal Form (1NF)

A relation is in First Normal Form when all attributes contain atomic values, each cell stores only one value, and there are no repeating groups. In this system, a user may have multiple phone numbers and a student enrollment may contain multiple courses, so those multi-value situations were separated into their own relations.

Relation	1NF Check	Reason
User	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
Student	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
Faculty	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
Department	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
Program	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
Batch	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
Batch_Program	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
Academic_Session	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
Semester	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
Section	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
Course	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
Teacher_Assignment	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
Enrollment	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
Registered_Course	In 1NF	One registered course is stored per row; courses are not repeated inside Enrollment.
Assessment	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
Submission	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
Evaluation	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
User_Phone	In 1NF	One phone number is stored per row; phone numbers are not repeated inside User.

Activity_Log	In 1NF	Attributes are atomic and no repeating group is stored in the relation.
--------------	--------	---

1NF conversion steps applied:

Step 1: User phone numbers were separated from User into User_Phone so that multiple phone numbers are stored as multiple rows instead of Phone1, Phone2, and Phone3 columns.

Step 2: Registered courses were separated from Enrollment into Registered_Course so that each course registration is stored as a separate row.

Step 3: Assessment, Submission, and Evaluation attributes were kept as single-value fields such as portal status, file path, marks, feedback, and generated date.

3.3.2. Second Normal Form (2NF)

A relation is in Second Normal Form when it is already in 1NF and every non-key attribute depends on the complete primary key. Partial dependency usually appears in composite-key tables when an attribute depends on only part of the key. The final design avoids this by using proper associative relations and by placing relationship-specific attributes in the relation where they fully belong.

Relation	2NF Check	Reason
User	In 2NF	No partial dependency exists; non-key attributes depend on the relation key.
Student	In 2NF	No partial dependency exists; non-key attributes depend on the relation key.
Faculty	In 2NF	No partial dependency exists; non-key attributes depend on the relation key.
Department	In 2NF	No partial dependency exists; non-key attributes depend on the relation key.
Program	In 2NF	No partial dependency exists; non-key attributes depend on the relation key.
Batch	In 2NF	No partial dependency exists; non-key attributes depend on the relation key.
Batch_Program	In 2NF	Expected_Graduation depends on the batch-program combination, not only on Batch or only on Program.
Academic_Session	In 2NF	No partial dependency exists; non-key attributes depend on the relation key.
Semester	In 2NF	No partial dependency exists; non-key attributes depend on the relation key.
Section	In 2NF	No partial dependency exists; non-key attributes depend on the relation key.
Course	In 2NF	No partial dependency exists; non-key attributes depend on the relation key.
Teacher_Assignment	In 2NF	Relationship-specific attributes depend on the complete relation represented by the table.
Enrollment	In 2NF	Relationship-specific attributes depend on the complete relation represented by the table.

Registered_Course	In 2NF	Relationship-specific attributes depend on the complete relation represented by the table.
Assessment	In 2NF	No partial dependency exists; non-key attributes depend on the relation key.
Submission	In 2NF	No partial dependency exists; non-key attributes depend on the relation key.
Evaluation	In 2NF	No partial dependency exists; non-key attributes depend on the relation key.
User_Phone	In 2NF	No partial dependency exists; non-key attributes depend on the relation key.
Activity_Log	In 2NF	No partial dependency exists; non-key attributes depend on the relation key.

2NF conversion steps applied:

Step 1: The Batch and Program relationship was separated from the Batch table because a batch label can be connected with many programs.

Step 2: Expected_Graduation was placed in Batch_Program because it depends on both Batch and Program together.

Step 3: Enrollment, Registered_Course, and Teacher_Assignment were kept as separate relations so that their attributes depend on the complete academic relationship.

3.3.3. Third Normal Form (3NF)

A relation is in Third Normal Form when it is already in 2NF and no non-key attribute depends on another non-key attribute. This removes transitive dependency. In this project, several refinements were made to ensure that every non-key attribute depends directly on the primary key of its own relation.

Original Issue	Dependency Problem	3NF Fix Applied	Result
Batch stored Program_ID	One batch label can contain many programs, so Program_ID does not belong directly in Batch.	Removed Program_ID from Batch and introduced Batch_Program.	[x] Batch and Program are correctly separated.
Expected_Graduation in Batch	Expected graduation depends on both Batch and Program.	Moved Expected_Graduation to Batch_Program.	[x] Attribute is stored in the correct relation.
Student Current_Semester had	Current semester can be derived from Enrollment and Semester history.	Removed Current_Semester from Student.	[x] Derived data is not stored redundantly.
Student could have Section_ID	Section can change across sessions and semesters.	Stored Section_ID in Enrollment instead of Student.	[x] Section history is preserved.
Semester had year/date data	Year and dates describe actual academic sessions, not semester level.	Kept Semester as level only and used Academic_Session for dates.	[x] Semester and Session meanings are separate.
Registered_Course repeated Course_ID	Teacher_Assignment identifies the course being taught.	Kept Teacher_Assignment_ID in Registered_Course and	[x] Course data is not repeated.

		avoided Course_ID.	repeated	
--	--	-----------------------	----------	--

3NF conversion steps applied:

Step 1: Batch was converted into a pure cohort relation - Batch now stores only Batch_ID, Batch_Name, Start_Year, and Batch_Status.

Step 2: Batch_Program was introduced - The many-to-many relationship between Batch and Program is resolved through Batch_Program, and Expected_Graduation is stored there.

Step 3: Student was cleaned - Student stores stable identity and membership information only. Current_Semester and Section_ID are not stored in Student.

Step 4: Enrollment became the academic history table - Enrollment stores Student_ID, Section_ID, Session_ID, Semester_ID, Is_Repeat, status, start date, and end date.

Step 5: Registered_Course was simplified - Registered_Course connects Enrollment with Teacher_Assignment. Submission uses Registered_Course so that the submitted file belongs to the exact enrolled course and teacher assignment.

Step 6: Semester and Academic_Session were separated - Semester stores academic level, while Academic_Session stores Spring/Fall, academic year, start date, and end date.

Final normalization status:

Normal Form	Status	Summary
1NF	Achieved	All attributes are atomic; repeating groups were separated into User_Phone and Registered_Course.
2NF	Achieved	All non-key attributes depend on the complete key or the complete relationship represented by the relation.
3NF	Achieved	Transitive dependencies were removed through Batch_Program, Academic_Session, Enrollment, and cleaned Student/Registered_Course design.

3.3.4. Final 3NF Relations

USER(

User_ID PK, First_Name, Last_Name, Email UNIQUE, Password_Hash, Role, Gender, Account_Status, Created_At)

DEPARTMENT(

Department_ID PK, Department_Name, Department_Code UNIQUE)

PROGRAM(

Program_ID PK, Department_ID FK, Program_Name, Program_Code UNIQUE, Degree_Level, Total_Semester)

BATCH(

Batch_ID PK, Batch_Name, Start_Year, Batch_Status)

BATCH_PROGRAM(

Batch_Program_ID PK, Batch_ID FK, Program_ID FK, Expected_Graduation)

ACADEMIC_SESSION(

Session_ID PK, Session_Name, Academic_Year, Start_Date, End_Date)

SEMESTER(

Semester_ID PK, Semester_No)

FACULTY(
Teacher_ID PK, User_ID FK UNIQUE, Department_ID FK, Designation)

STUDENT(
Student_ID PK, User_ID FK UNIQUE, Batch_Program_ID FK, Registration_No UNIQUE, Student_Status)

USER_PHONE(
Phone_ID PK, User_ID FK, Phone_No)

ACTIVITY_LOG(
Log_ID PK, User_ID FK, Activity_Description, Time)

COURSE(
Course_ID PK, Department_ID FK, Course_Code UNIQUE, Course_Title, Credit_Hours)

SECTION(
Section_ID PK, Batch_Program_ID FK, Section_Name, Capacity)

TEACHER_ASSIGNMENT(
Teacher_Assignment_ID PK, Teacher_ID FK, Course_ID FK, Semester_ID FK, Section_ID FK, Assigned_Date)

ENROLLMENT(
Enrollment_ID PK, Student_ID FK, Section_ID FK, Session_ID FK, Semester_ID FK, Is_Repeat, Enrollment_Status, Enrollment_Start_Date, Enrollment_End_Date)

REGISTERED_COURSE(
Registered_Course_ID PK, Enrollment_ID FK, Teacher_Assignment_ID FK, Registration_Status)

ASSESSMENT(
Assessment_ID PK, Teacher_Assignment_ID FK, Session_ID FK, Title, Description, Assessment_Type, Total_Marks, Created_At, Deadline, Portal_Status, Class_Report_Path, Class_Report_Generated_At, Class_Report_Status)

SUBMISSION(
Submission_ID PK, Assessment_ID FK, Registered_Course_ID FK, File_Name, File_Path, Upload_Time, Submission_Status, UNIQUE(Assessment_ID, Registered_Course_ID))

EVALUATION(
Evaluation_ID PK, Submission_ID FK UNIQUE, Obtained_Marks, Percentage, Evaluated_At, Evaluation_Status, Feedback, Student_Report_Path, Student_Report_Generated_At, Student_Report_Status)

CHAPTER 4: IMPLEMENTATION

4.1. LANGUAGE/FRAMEWORK:

The Fair & Unbiased Assessment System is implemented as a database-driven web application. The implementation uses Python with Flask for backend routing and business logic, MySQL for relational data storage, HTML templates with Jinja for dynamic pages, and CSS/JavaScript for the user interface behavior. This stack was selected because it supports structured database operations, clear separation of routes, and simple integration with MySQL for academic workflows such as enrollment, teacher assignment, assessment portals, submissions, evaluations, and report metadata.

4.1.1. Database: MySQL

MySQL is used as the primary relational database management system. It stores users, departments, programs, batches, sections, courses, enrollments, registered courses, assessments, submissions, evaluations, and report path information. The database is normalized and uses primary keys, foreign keys, unique constraints, views, stored procedures, functions, and triggers to protect data consistency.

Reasons for Selection:

- Supports normalized relational database design.

- Provides referential integrity using primary and foreign keys.

- Supports stored procedures, functions, views, and triggers required in the milestone.

- Handles joins between academic entities such as Enrollment, Registered_Course, Teacher_Assignment, Assessment, Submission, and Evaluation.

4.1.2. Backend: Python with Flask

The backend of the system is developed using Python and Flask. Flask routes are separated into Admin, Faculty, Student, and Authentication route files. The backend handles login sessions, role-based access, form submission, validation, database procedures, and secure data retrieval for dashboards and detail pages.

Reasons for Selection:

- Lightweight framework suitable for academic web applications.
- Simple routing structure for separate Admin, Faculty, and Student modules.
- Easy integration with MySQL through a database connection file.
- Supports server-side validation before calling stored procedures and database operations.

4.1.3. Frontend: HTML, Jinja Templates, CSS, and JavaScript

The graphical interface is developed using HTML templates rendered through Flask and Jinja. CSS is used for layout and design, while JavaScript is used where dynamic behavior is needed, especially in the enrollment page where student information and available assigned courses are loaded based on the entered registration number, selected section, and semester.

Reasons for Selection:

- Jinja templates allow database records to be displayed dynamically.
- CSS provides a consistent design across Admin, Faculty, and Student screens.
- JavaScript improves usability by loading dependent enrollment data without unnecessary page changes.
- The template-based approach is suitable for a Flask-based university system.

4.1.4. Authentication and Role-Based Access

The system stores user login information in the User table and separates users according to roles such as Admin, Faculty, and Student. After successful login, Flask sessions are used to control access so that Admin users access administrative pages, Faculty users access assigned course and assessment pages, and Student users access their current enrolled courses and submissions.

Reasons for Selection:

- Prevents unauthorized access to role-specific pages.
- Keeps common user profile information in one User relation.
- Allows Student and Faculty information to remain in separate subtype relations.
- Supports clean login flow for three system roles.

4.2. DATABASE CONNECTIVITY:

The system follows a Flask-based web application structure where the frontend templates do not directly communicate with the database. User actions are sent to Flask routes, and the routes communicate with the MySQL database using a database connection helper. This keeps database credentials and SQL operations on the backend side and allows each route to apply validation before inserting, updating, or retrieving records.

4.2.1. Technologies Used:

Component	Technology
Frontend	HTML, Jinja Templates, CSS, JavaScript

Backend	Python with Flask
Database	MySQL
Database Driver	mysql-connector-python
API Style	Flask routes and JSON API endpoints
Authentication	Role-based login using Flask sessions

4.2.2. Database Connection Mechanism:

The database connection is maintained through the db.py file. Each route obtains a database connection, creates a cursor with dictionary output, executes SQL queries or stored procedures, commits the transaction when changes are successful, and rolls back the transaction if an error occurs. This mechanism allows database operations to remain consistent and easier to manage.

Connection file:

File	Purpose
db.py	Creates and returns the MySQL database connection used by all route files.
app.py	Registers route blueprints and starts the Flask application.
routes/admin_routes.py	Uses views and stored procedures for Admin data management.
routes/faculty_routes.py	Handles assigned courses, assessment portals, submissions, evaluations, and reports.
routes/student_routes.py	Handles student dashboard, course detail, upload, re-upload, and report viewing.

4.2.3. Communication Flow:

A user performs an action from the web interface, such as adding a student, creating an enrollment, viewing assigned courses, uploading a submission, or evaluating a submission.

The HTML form or JavaScript request sends the data to the related Flask route.

The route validates required fields and checks user role through session data.

The route executes SQL queries, views, functions, or stored procedures using the MySQL connection.

The database applies constraints and triggers to protect data consistency.

The route commits successful changes or rolls back errors and displays a message to the user.

The updated template is rendered again with the latest database data.

4.2.4. Database Server Configuration:

The database server runs on MySQL and stores all system data related to users, academic structure, enrollments, teacher assignments, assessments, submissions, evaluations, and report metadata. The system connects to the database using the configured host, user, password, and database name. The selected database for implementation is fair_assessment_system.

4.3. STORED PROCEDURES AND FUNCTIONS:

The implementation uses database objects to reduce repetition in the backend and to enforce important checks at the database level. Views are used for repeated display queries, functions are used for reusable checks, stored procedures are used for controlled insert/update/delete operations, and triggers are used to protect submission and evaluation rules.

#	Object Name	Object Type	Purpose / Use in System
1	v_admin_dashboard_counts	VIEW	Returns dashboard summary counts for students, faculty, courses, enrollments, teacher assignments, active users, inactive users, and departments.
2	v_admin_students	VIEW	Provides student records with user, department, program, and batch information for admin lists and detail pages.
3	v_admin_faculty	VIEW	Provides faculty records with user and department information for admin lists and detail pages.
4	v_admin_courses	VIEW	Provides course information with department name and department code.
5	v_admin_teacher_assignments	VIEW	Provides teacher assignment details by joining faculty, course, section, semester, batch, program, and department data.
6	v_admin_enrollments	VIEW	Provides enrollment records with student, academic session, semester, section, batch, program, and department information.
7	v_admin_enrollment_registered_courses	VIEW	Shows all registered courses linked with an enrollment along with course and teacher details.
8	fn_email_exists	FUNCTION	Checks duplicate email values during add and update operations.
9	fn_registration_exists	FUNCTION	Checks duplicate student registration numbers during add and update operations.
10	fn_course_code_exists	FUNCTION	Checks duplicate course codes before adding a new course.
11	fn_section_available_seats	FUNCTION	Calculates remaining seats in a section for a selected session and semester.
12	fn_current_session_id	FUNCTION	Returns the current academic session based on the current date, with a fallback to the latest started session.
13	fn_student_already_enrolled	FUNCTION	Checks whether a student already has an active enrollment in the same section, session, and semester.
14	sp_add_student / sp_update_student	PROCEDURE	Adds or updates student records while validating email, registration number, and batch-program selection.
15	sp_add_faculty / sp_update_faculty	PROCEDURE	Adds or updates faculty records while validating email and department information.
16	sp_add_course	PROCEDURE	Adds a course after validating department, course code, title, and credit hours.
17	sp_add_teacher_assignment	PROCEDURE	Creates a teacher assignment after validating teacher, course, semester, section, duplicate assignment, and department match.
18	sp_add_enrollment	PROCEDURE	Creates an enrollment after validating active student status, section-program match, duplicate enrollment, and section capacity.
19	sp_add_registered_course	PROCEDURE	Registers selected courses under an enrollment after validating section, semester, duplication, and repeat-course rules.
20	sp_delete_student_safe / sp_delete_faculty_safe / sp_delete_course_safe / sp_delete_teacher_assignment_safe	PROCEDURE	Prevents unsafe deletion when academic history or dependent records already exist.
21	trg_prevent_submission_when_portal_closed	TRIGGER	Blocks new submissions when the related assessment portal is closed.

22	trg_prevent_submission_update_when_portal_closed	TRIGGER	Blocks re-upload or submission update when the assessment portal is closed.
23	trg_calculate_evaluation_percentage_insert/update	TRIGGER	Automatically calculates percentage values from obtained marks and assessment total marks.
24	trg_prevent_duplicate_active_enrollment	TRIGGER	Prevents duplicate active enrollment for the same student, section, session, and semester.

4.4. INTERFACES:

This section will contain screenshots of the implemented system. The screenshot spaces are intentionally left empty at this stage so that final interface images can be inserted after GUI polishing and final testing. The headings below represent the major pages of the Fair & Unbiased Assessment System.

4.4.1. Admin Dashboard

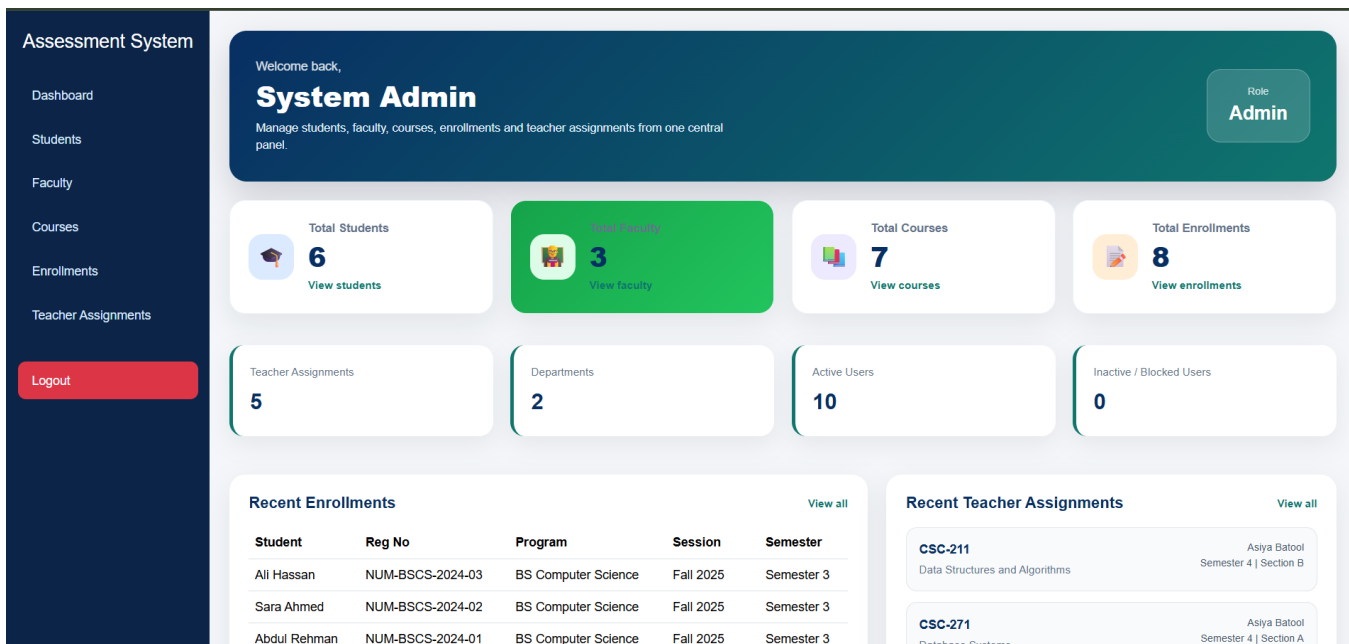


Figure 1: Admin Dashboard

The admin dashboard displays overall academic and user statistics using database views.

4.4.2. Student Management Page

Assessment System

Dashboard

Students

Faculty

Courses

Enrollments

Teacher Assignments

Logout

Students Management

Admin Panel

Students

Add New Student

Search by registration number, name, department, or program

Search

#	Name	Registration No	Department	Program	Batch	Account Status	Student Status	Action
1	Abdul Rehman	NUM-BSCS-2024-01	Computer Science	BS Computer Science	Batch 2024	Active	Active	View / Edit
2	Sara Ahmed	NUM-BSCS-2024-02	Computer Science	BS Computer Science	Batch 2024	Active	Active	View / Edit
3	Ali Hassan	NUM-BSCS-2024-03	Computer Science	BS Computer Science	Batch 2024	Active	Active	View / Edit
4	Ayesha Malik	NUM-BSCS-2024-04	Computer Science	BS Computer Science	Batch 2024	Active	Active	View / Edit
5	Bilal Khan	NUM-BSCS-2024-05	Computer Science	BS Computer Science	Batch 2024	Active	Active	View / Edit
6	Mehroz Ali Khan	NUM-BSCS-2024-34	Computer Science	BS Computer Science	Batch 2024	Active	Enrolled	View / Edit

Figure 2: Student Management Page

This page is used to view, add, update, and manage student records.

4.4.3. Faculty Dashboard

Assessment System

Dashboard

My Courses

Logout

Faculty Dashboard

Rafay Khan

View assigned courses, create assessments, review submissions and evaluate student work.

Session
Spring 2026

Assigned Courses
1
View courses

Total Assessments
4
Manage

Pending Evaluations
3
Check submissions

My Assigned Courses

CSC-211 - Data Structures and Algorithms

BS Computer Science

Batch 2024 | Section A | Semester 4

5 enrolled students

Open Course

View all

Recent Assessments

Figure 3: Faculty Dashboard

APPENDIX: AI USAGE AND PROMPTS

#	Purpose	Prompt Used
1	SQL Script Generation	"Generate INSERT statements with sample data for all tables in the university timetable database."
2	Frontend Integration	"I already have frontend in React JS, how to connect it to MySQL?"
3	Database Triggers	"Write a trigger that prevents venue scheduling conflicts when inserting timetable records."
4	Database Functions	"Create a MySQL function to check faculty availability for a given day and time slot."
5	React Frontend Development	"Generate a React component for timetable management with add, edit, delete, and publish functionality."
6	Report Revision	"Generate version history descriptions for Version 1.0 and Version 2.0 of the project report."
7	Database Connectivity	"Explain how to connect a React frontend to MariaDB database using Node.js and Express."
9	Debugging	"When I try to change password of Ahmad Ali, it says failed to change passowrd."
10	Documentation	"Generate report documentation for database connectivity and database objects."
11	Frontend Enhancement	"Multiple schedules can be on same slot on same day without any venue/instructor conflict, but when I add it, timetable grid on admin side shows only one entry in grid cell. Convert this into prompt for Cursor."

